

ENTERPRISE ARCHITECTURE BLUEPRINT

Cloud Accounting & Business Management Platform

(Codename "Ledger" - Zoho Books-class product)

VOLUME 6 OF 9

SECURITY ARCHITECTURE

Threat Model - IAM & RBAC - Encryption - Secrets - Audit Trail - Compliance

Field	Value
Document Status	Draft - v0.1 - Phase 6 of 9
Prepared For	caldim
Prepared By	Claude (Enterprise Architecture Assistant)
Date	13 July 2026
Classification	Internal - Pre-Development Planning - Sensitive
Upstream Dependency	Vol. 1 BC-20, Section 10 Compliance; Vol. 2 CTX-01 Identity & Tenant; Vol. 3 Section 11 RLS multi-tenancy; Vol. 4 Section 7 RLS implementation; Vol. 5 Section 4.2 PCI scope, Section 9 credential store
Downstream Volumes	Vol. 7 Infrastructure & Deployment; Vol. 8 Testing & QA; Vol. 9 UI/UX & API Design

Table of Contents

1. Introduction & Scope

1.1 Purpose

This volume formalizes the security posture implied but not fully specified in earlier volumes: the threat model, identity and access management design, encryption and secrets management standards, API/network security, defense-in-depth for multi-tenant isolation, audit-trail tamper-evidence, and the mapping from all of the above to concrete compliance frameworks (SOC 2, GDPR, PCI, and HIPAA where contingently applicable per Vol. 1 Section 10).

1.2 Security Principles Inherited From Earlier Volumes

- Vol. 1 Principle P4 (auditability): every financial mutation is append-only and traceable - Section 9 of this volume defines how that trail is made tamper-evident.
- Vol. 3 Section 11 (shared schema + RLS): this volume's Section 8 treats RLS as one layer of defense-in-depth, not the only control.
- Vol. 5 Principle I3 (webhook signature verification) and Section 4.2 (PCI SAQ-A via tokenization): this volume's Section 5-6 generalize those patterns platform-wide.
- Vol. 2 Section 5 API Gateway as the single ingress point: this volume's Section 7 defines what runs at that boundary.

2. Threat Model

Threats are organized using STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) against the highest-value assets identified across Vol. 1-5: tenant data isolation, financial transaction integrity, and stored credentials/tokens.

2.1 Asset Inventory

Asset	Why It Matters	Primary Owning Volume
Tenant financial data (ledger, invoices, bills)	Core business value; unauthorized disclosure or tampering is catastrophic for trust and regulatory standing	Vol. 4
Authentication credentials & session tokens	Compromise enables full account takeover	Section 3, this volume
Third-party API credentials (bank tokens, payment gateway keys)	Compromise enables financial fraud against customers' connected accounts	Vol. 5 Section 9; Section 6, this volume
Audit trail / change log	Regulatory and forensic value; must itself be tamper-evident	Section 9, this volume
Tenant configuration (roles, custom fields, workflow rules)	Misconfiguration or tampering can silently weaken a tenant's own controls	Vol. 2 CTX-14

2.2 STRIDE Analysis by Flow

Flow	Threat (STRIDE)	Mitigation
User login	Spoofing (credential stuffing, phishing)	MFA (Section 3.3), rate-limited login attempts, breached-password screening
API request with JWT	Spoofing (forged/stolen token), Tampering (modified claims)	Short-lived signed JWTs (Section 3.2), signature verification on every request at the Gateway
Cross-tenant data access	Information Disclosure, Elevation of Privilege	RLS (Vol.4 Section 7) + application-layer tenant check (Section 8 defense-in-depth), automated leakage tests (Vol.4 Section 7, Vol.8)
Financial mutation (e.g. void an invoice)	Repudiation (user denies action), Tampering	Append-only ledger (Vol.1 P4), signed audit log entries (Section 9), correlation ID linking action to authenticated user
Inbound webhook (bank, payment gateway, e-commerce)	Spoofing (forged webhook), Tampering	Signature verification (Vol.5 Principle I3), IP allowlisting where the provider supports it
Stored bank/payment credentials	Information Disclosure if database or backups are exposed	Envelope encryption at rest (Section 5.2), secrets never logged (Section 6.3)
Public API / webhook platform (Vol.5 Section 7)	Denial of Service (abusive third-party integrator)	Per-tenant/per-key rate limiting (Vol.3 Section 7.1, Vol.5 Section 7.2)
Custom role / permission configuration (Vol.1 BC-17)	Elevation of Privilege via misconfigured custom role	Permission matrix has a maximum ceiling per base role (Section 4.2) that a custom role cannot exceed

3. Identity & Access Management

3.1 Authentication Model

Mechanism	Detail
Primary auth	Email + password (Argon2id hashed, never reversible) or SSO/SAML/OIDC for Enterprise-edition tenants (Vol.1 Section 9 entitlement)
Session tokens	Short-lived JWT access token (15 min) + longer-lived rotating refresh token (httpOnly, secure cookie for web; secure storage for mobile), issued by CTX-01 Identity & Tenant
MFA	TOTP-based MFA available at all editions, enforced by tenant policy at Professional/Enterprise; required unconditionally for any user holding an Admin or Accountant role (Vol.1 PER-01, PER-02)
Portal auth (customers/vendors, Vol.1 PER-09/PER-10)	Separate, more restricted auth flow (magic-link or lightweight password) scoped only to the portal API surface (Vol.2 Section 11 Collaboration & Portals), never granted internal-staff JWT scopes
Service-to-service auth (post-extraction, Vol.2 Section 2.1)	mTLS or signed service tokens between extracted services (Payments, Integration Gateway) and the core application, not end-user JWTs

3.2 JWT Claim Structure

JWT claims (access token)

```
{
  "sub": "usr_4471",
  "tenantId": "org_8f2a1c",
  "roles": ["accountant"],
  "permissions": ["receivables:write", "ledger:read", "reporting:read"],
  "mfaVerified": true,
  "iat": 1752400000,
  "exp": 1752400900
}
```

The tenantId claim is what Vol. 3 Section 11.2's tenant-context middleware uses to set the RLS session variable - it is set only by CTX-01 at token issuance and is never accepted from a client-supplied header or parameter (Vol. 3 Section 5 tenant-scoping convention).

3.3 Role Model (Base Roles)

Role	Maps to Persona (Vol.1)	Default Scope
Owner/Admin	PER-01	Full access to all modules within the tenant's edition entitlement, including user/role management
Accountant	PER-02	Full read/write on Ledger, Receivables, Payables, Banking, Reporting; no user management
External Accountant	PER-03	Same as Accountant, but access is granted per-tenant via an explicit invitation record, not tenant membership
Sales Staff	PER-04	Read/write on Receivables only; read-only on Inventory for stock checks

Role	Maps to Persona (Vol.1)	Default Scope
Purchasing Staff	PER-05	Read/write on Payables only; read-only on Inventory
Inventory Manager	PER-06	Read/write on Inventory; read-only on Receivables/Payables
Project Manager	PER-07	Read/write on Projects; read-only on Receivables for billing
Payroll Admin	PER-08	Read/write on Payroll only (R3)
Customer (portal)	PER-09	Read-only on own invoices/quotes; write on payments and comments, scoped to the portal API surface only
Vendor (portal)	PER-10	Read-only on own POs/bills; write on bill submission, scoped to the portal API surface only

4. Authorization Model

4.1 Permission Structure

Permissions are expressed as module:action pairs (e.g. receivables:write, ledger:read), matching the bounded-context/API-resource ownership fixed in Vol. 2 Section 11. Every API endpoint declares its required permission(s) at the controller level (Vol. 3 Section 4), and the Gateway/middleware layer rejects any request whose JWT permissions do not include it - authorization is enforced centrally, not re-implemented per module.

4.2 Custom Roles (Vol. 1 BC-17) - Ceiling Rule

Tenant administrators may define custom roles (Enterprise edition, Vol. 1 Section 9) by composing permissions, but a custom role can never grant a permission the composing admin does not themselves hold, and no custom role can exceed the permission ceiling of the closest base role it resembles. This prevents the privilege-escalation threat identified in Section 2.2 where a misconfigured custom role could otherwise grant unintended access.

4.3 Resource-Level Authorization

Role-based permissions grant module-level access; resource-level checks (e.g. "can this Sales Staff user view this specific customer's invoices" under a tenant that restricts by sales territory) are a Release 2/3 capability layered on top of the base RBAC model, implemented as an additional policy check in the domain service layer (Vol. 2 Section 6.1 component pattern), not by modifying the JWT/role model itself.

5. Data Protection & Encryption

5.1 Encryption in Transit

- TLS 1.2+ enforced on every external-facing endpoint (web app, API gateway, portal, public API) - no plaintext HTTP, including internal health-check endpoints exposed beyond the deployment network.
- Internal service-to-service traffic (post-extraction services, Vol.2 Section 2.1) uses TLS as well, moving toward mTLS as the service count grows (Vol.7 to finalize the service-mesh decision).

5.2 Encryption at Rest

Data Category	Encryption Approach
Database (PostgreSQL, Vol.4)	Full-disk/volume encryption at the infrastructure layer (Vol.7) plus column-level envelope encryption for the specific high-sensitivity fields below
Bank/payment provider tokens (Vol.5 Section 9 credential store)	Envelope encryption: each secret encrypted with a data encryption key (DEK), DEKs themselves encrypted by a master key held in a managed KMS (Section 6.1) - the database never stores a usable plaintext credential even if the DB itself is exfiltrated
PII fields (customer/vendor tax ID, personal address on portal accounts)	Column-level encryption using the same envelope pattern, queryable only through the owning module's domain service (Vol.2 Section 6.1), never via raw SQL by another module
Object storage (documents, exports, Vol.3 Section 2.7)	Server-side encryption at the storage provider level (SSE-S3/KMS-equivalent)
Backups (Vol.7 to detail)	Encrypted with the same at-rest standard as the primary store; backup encryption keys rotated independently of primary DEKs

6. Secrets & Credential Management

6.1 Key Management

A managed Key Management Service (cloud-provider KMS, per Vol. 3 Section 2.1's eventual cloud-provider confirmation) holds the master keys for the envelope-encryption pattern in Section 5.2. Application code never handles a master key directly - it calls the KMS to wrap/unwrap DEKs, and master key rotation is a KMS-native operation that does not require re-encrypting all data at rest.

6.2 Secrets Store (formalizing Vol. 5 Section 9's Credential Store)

Secret Type	Storage	Access Pattern
Tenant bank/payment provider tokens	Encrypted columns in the Integration Gateway's own schema (Vol.4 Section 1.2 ownership rule), envelope-encrypted per Section 5.2	Decrypted only inside the Integration Gateway service at the moment of use; never returned in any API response, even to the owning tenant's admin
Platform-level API keys (KMS credentials, Kafka auth, database connection secrets)	A dedicated secrets manager (e.g. cloud-provider Secrets Manager/Vault-class), not environment variables in plaintext config files	Injected into the runtime environment at deploy time (Vol.7); rotated on a defined schedule and immediately on suspected compromise
Webhook signing secrets (Vol.5 Section 7.1)	Generated per subscription, stored encrypted, never displayed again after initial creation (only a masked/truncated form shown in the UI)	Verified server-side only; never sent to any client

6.3 Secrets Hygiene Rules

1. Secrets are never written to application logs (Vol.3 Section 6) - log statements are code-reviewed and CI-scanned for accidental secret interpolation.
2. Secrets are never committed to the monorepo (Vol.3 Section 3.1) - a pre-commit and CI secret-scanning gate blocks any match against common credential patterns.
3. Every credential has a defined owner and rotation cadence, tracked in an internal secrets inventory (Vol.7 operational runbook).

7. API & Network Security

7.1 API Gateway Security Controls

Control	Detail
Authentication enforcement	Every route requires a valid JWT unless explicitly marked public (e.g. login, signup, webhook receivers which use signature verification instead, Vol.5 Section 3.4/7.1)
Rate limiting	Per-tenant and per-API-key limits (Vol.3 Section 7.1, Vol.5 Section 7.2), plus a global anti-abuse limit per source IP for unauthenticated endpoints (login, signup)
Input validation	Schema validation (DTOs, Vol.3 Section 3.3) rejects malformed requests before they reach domain logic; parameterized queries only, no dynamic SQL (prevents injection)
WAF / edge protection	A web application firewall at the edge (Vol.7 to select concrete provider) filters common attack patterns (SQLi, XSS payloads, known bad IP ranges) before traffic reaches the Gateway
CORS policy	Web app and portal origins explicitly allowlisted; no wildcard CORS on any authenticated endpoint
Security headers	HSTS, X-Content-Type-Options, Content-Security-Policy set on all HTTP responses from the web app and portal

7.2 Network Segmentation

The database, cache, and event bus (Vol. 3 Section 2) are not directly reachable from outside the private network - only the API Gateway and background workers can reach them. The Integration Gateway service, which makes outbound calls to third parties (Vol. 5), sits in a segment with restricted outbound rules (allowlisted destination domains only) to limit blast radius if it is ever compromised. Concrete VPC/subnet design is finalized in Volume 7.

8. Multi-Tenant Isolation: Defense in Depth

Vol. 4 Section 7 established PostgreSQL RLS as the primary tenant-isolation control. This section adds the layers around it, since a single control point is a single point of failure for the platform's highest-severity risk (cross-tenant data disclosure, Section 2.1).

Layer	Control
1. Database (primary)	Row-Level Security policy on every tenant-scoped table (Vol.4 Section 7), fail-closed if tenant context is unset
2. Application middleware (secondary)	Every domain service call is passed an explicit tenantId derived from the JWT (Section 3.2), independently checked against the entity being accessed before any RLS-protected query even runs
3. Cache layer (tertiary)	Tenant-prefixed cache keys (Vol.3 Section 7.2) enforced by a shared cache utility that cannot be bypassed by individual modules
4. Automated testing (verification)	Cross-tenant-leakage test suite (Vol.4 Section 7, Vol.8) runs on every CI build against every RLS-protected table
5. Operational monitoring (detection)	Anomaly alerts (Vol.7) for query patterns that touch an unusual number of distinct tenant_id values in a short window, which would indicate an RLS bypass or a compromised service credential

9. Audit Logging & Tamper-Evidence

Formalizes Vol. 1 BC-20's audit trail requirement and Vol. 1 Principle P4's auditability principle into a concrete design.

9.1 What Is Audited

- Every financial-entity state change (invoice sent/voided, journal posted, bill approved) - already captured as domain events (Vol.2 Section 10) and additionally written to a dedicated append-only audit_log table.
- Every authentication event (login success/failure, MFA challenge, password reset, role change) - security-relevant even where no financial entity changed.
- Every permission/role configuration change (Section 4.2) - who granted what to whom, and when.
- Every access to a decrypted secret (Section 6.2) - logged with the requesting service identity, not just the tenant.

9.2 Tamper-Evidence Mechanism

Audit log entry - hash-chained for tamper evidence

```
{
  "id": "audit_9f21...",
  "tenantId": "org_8f2a1c",
  "actorUserId": "usr_4471",
  "action": "invoice.voided",
  "resourceId": "inv_0042",
  "timestamp": "2026-07-13T10:22:31.104Z",
  "correlationId": "c7f1e2-9b3a-4d10",
  "previousEntryHash": "3a7f...",
  "entryHash": "sha256(previousEntryHash + serialized(this entry))"
}
```

Each audit entry includes a hash of the previous entry, forming a per-tenant hash chain; any retroactive edit or deletion of a past entry breaks the chain and is detectable by a periodic integrity-verification job. This does not require exotic infrastructure (e.g. a blockchain) - the hash chain plus the append-only database constraint (Vol.4 Section 2, D2/D6) is sufficient tamper-evidence for the compliance bar this platform targets (Section 10).

10. Compliance Control Mapping

Maps this volume's controls to the compliance scope Vol. 1 Section 10 defined, so a future SOC 2 readiness assessment or customer security questionnaire can be answered by reference rather than re-derivation.

Framework	Representative Control	Satisfied By
SOC 2 (Security)	Logical access controls	Section 3 IAM, Section 4 Authorization Model
SOC 2 (Security)	Encryption of data at rest and in transit	Section 5
SOC 2 (Availability)	Rate limiting and DoS protection	Section 7.1
SOC 2 (Processing Integrity)	Tamper-evident change history	Section 9
GDPR	Data subject right to erasure vs. financial retention conflict	Vol.4 Section 10 (retention policy) resolves precedence; Section 5.2 field-level encryption supports crypto-shredding of PII where full deletion is legally blocked
GDPR	Data processing records / lawful basis	Out of this volume's scope - a legal/compliance function tracks this; the architecture supports it via the audit trail (Section 9)
PCI DSS (SAQ-A)	No storage of raw cardholder data	Vol.5 Section 4.2 tokenization approach; verified by never persisting card PANs anywhere in this platform's own schemas (Vol.4)
HIPAA (contingent, Vol.1 Section 10)	Access controls and audit logging for PHI	Sections 3, 4, 9 - only activated as a control set if a healthcare-vertical edition is pursued, per Vol.1's contingent scoping

11. Security Monitoring & Incident Response (Summary)

Full operational runbooks belong to Volume 7; this section fixes the architectural hooks Volume 7's monitoring stack must consume.

Signal	Source	Response
Repeated failed login / credential stuffing pattern	Auth service (Section 3.1)	Automatic account lockout + alert; feeds Section 3.3 MFA enforcement escalation
RLS bypass indicator (Section 8, layer 5)	Database query monitoring	Page on-call immediately - highest severity, treated as a potential cross-tenant breach until ruled out
Audit hash-chain break (Section 9.2)	Periodic integrity job	Page on-call; freeze affected tenant's write access pending investigation
Webhook signature verification failure spike (Vol.5 Section 3.4)	Integration Gateway	Alert - may indicate a compromised or spoofed third-party integration
Anomalous secret-access pattern (Section 6.3)	Secrets manager audit log	Alert; correlate with the requesting service's recent deploy/change history

12. Security Testing Requirements

Volume 8 (Testing & QA) owns the full test strategy; this volume specifies the security-specific tests that must exist before Release 1 ships.

Test Type	Scope	Cadence
Cross-tenant leakage suite	Every RLS-protected table (Vol.4 Section 7, Section 8 layer 4)	Every CI build
SAST (static analysis)	Full monorepo, focused on the OWASP Top 10 and the secrets-hygiene rules (Section 6.3)	Every CI build
Dependency vulnerability scanning	All npm packages (Vol.3 Section 2.1 stack)	Every CI build + daily scheduled scan
DAST (dynamic analysis)	Public API surface and web app	Weekly against staging
Third-party penetration test	Full platform, with particular focus on the integration surfaces (Vol.5) and multi-tenant isolation (Section 8)	Before Release 1 GA, then annually
Webhook/API abuse testing	Rate limiting (Section 7.1), signature verification bypass attempts (Section 2.2)	Before Release 1 GA

13. Traceability & Next Phase

13.1 Traceability to Volumes 1-5

Vol. 6 Artifact	Traces To
Section 2 Threat Model	Vol. 1 Principle P4; Vol. 2 Section 5 Gateway boundary; Vol. 5 integration surfaces
Section 3 IAM / Section 4 Authorization	Vol. 1 Section 4 Personas (PER-01...PER-12); Vol. 1 Section 9 Editions
Section 5-6 Encryption/Secrets	Vol. 5 Section 9 Credential Store; Vol. 3 Section 2.1 stack
Section 8 Multi-Tenant Isolation	Vol. 3 Section 11, Vol. 4 Section 7 RLS
Section 9 Audit Trail	Vol. 1 BC-20, Principle P4
Section 10 Compliance Mapping	Vol. 1 Section 10 Compliance & Regulatory Scope

13.2 Open Questions for Sign-off

4. Confirm whether a formal SOC 2 Type II audit is a near-term business requirement (affects how rigorously Section 10's controls must be evidenced from Release 1 versus retrofitted later).
5. Confirm KMS/secrets-manager vendor preference once the cloud provider (Vol.3 Section 13.2 Q1) is confirmed.
6. Confirm resource-level authorization (Section 4.3) is acceptable as an R2/R3 capability rather than an MVP requirement.

13.3 Next Phase

On approval, Phase 7 begins: Infrastructure & Deployment Architecture (Volume 7) - CI/CD pipeline, environment topology, disaster recovery, observability, and the concrete sizing/scaling plan that operationalizes every architectural decision made in Volumes 1 through 6.